

California Housing Price Prediction

A Line-by-Line Code Walkthrough

Dataset: California Housing Prices (housing.csv) — Kaggle: camnugent/california-housing-prices

Pipeline: EDA → Preprocessing → Baseline Model → Model Selection (CV) → Tuning (GridSearchCV) → Final Evaluation → Inference

1. Setup

%pip install is a Jupyter "magic command" — it runs pip inside the current notebook kernel rather than a separate shell. The **-q** flag just suppresses the verbose install logs.

```
%pip install -q numpy pandas matplotlib seaborn scikit-learn
```

The imports aren't random — each one maps directly onto a stage of the ML workflow:

- **train_test_split** — splits data into train/test sets
- **KFold** — defines *how* to split training data into k folds for cross-validation
- **cross_validate** — runs a model through those folds and returns scores
- **GridSearchCV** — exhaustively tests every hyperparameter combination via CV
- **Pipeline** — chains preprocessing + model into one object, so `.fit()/predict()` run both steps in the right order automatically
- **ColumnTransformer** — applies different preprocessing to different column subsets (numeric vs categorical)
- **StandardScaler** — rescales numeric columns to mean 0, std 1
- **OneHotEncoder** — turns category strings into binary indicator columns
- **SimpleImputer** — fills in missing values

```
from sklearn.model_selection import train_test_split, KFold, cross_validate, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
```

Why Pipeline + ColumnTransformer matter: if you scaled or imputed the whole dataset before splitting, statistics like the median or mean would leak information from the test set into training. Wrapping everything in a Pipeline means each `.fit()` call (in CV or on `X_train`) recomputes imputer/scaler/encoder statistics only from the data it's given — that's what "avoiding data leakage" means in practice, not just a comment.

Models chosen and why each one is on the shortlist:

- **LinearRegression** — baseline, assumes a straight-line relationship
- **Ridge** — linear regression + L2 penalty; shrinks coefficients, helps with multicollinearity (relevant since the correlation heatmap shows room/population features are highly correlated)

- **Lasso** — linear regression + L1 penalty; can zero out useless features entirely
- **RandomForestRegressor** — many decision trees on bootstrapped samples, averaged
- **HistGradientBoostingRegressor** — sklearn's fast histogram-based boosting; trees built sequentially, each correcting the previous one's errors

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
```

Metrics:

- **MAE** — average absolute error, in dollars, robust to outliers
- **RMSE** — square root of the average squared error; punishes big misses harder — the project's primary metric
- **R²** — fraction of variance explained (1 = perfect, 0 = no better than guessing the mean, can go negative if the model is worse than that)

```
from sklearn.metrics import (
    mean_absolute_error,
    root_mean_squared_error,
    r2_score
)
```

Configuration block — small settings that pay off for the rest of the notebook:

- **pd.set_option("display.max_columns", None)** — don't truncate columns when printing wide DataFrames
- **pd.set_option("display.float_format", ...)** — print floats with 3 decimals everywhere
- **sns.set_theme(style="darkgrid")** — global plot styling (grid lines, cleaner look)
- **plt.rcParams.update({...})** — sets default font sizes for titles/labels/ticks once, instead of repeating them on every plot
- **RANDOM_STATE = 42** — fixes the random seed so splits and randomized models give identical results across runs (reproducibility)
- **CSV_PATH / TARGET_COL** — store the filename and target column name as variables, so the string is typed once and typos can't creep in later

```
RANDOM_STATE = 42
CSV_PATH = "housing.csv"
TARGET_COL = "median_house_value"
```

2. Load Data

```
df = pd.read_csv(CSV_PATH)

from google.colab import drive
drive.mount('/content/drive')

print("Data frame Shape :", df.shape)
df.head()
```

pd.read_csv loads the CSV into a DataFrame. The **drive.mount(...)** line is Colab-specific — it mounts Google Drive into the VM's filesystem at `/content/drive`.

Worth noticing: `drive.mount` runs *after* `read_csv`, so it isn't actually doing anything for this particular load (unless `housing.csv` was already cached from an earlier mount). It's probably a leftover from cell reordering rather than a bug that affects the result.

df.shape returns a (rows, columns) tuple. **df.head()** shows the first 5 rows — a quick sanity check that the file loaded correctly.

3. Exploratory Data Analysis (EDA)

```
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = df.select_dtypes(include=["object"]).columns.tolist()
```

select_dtypes filters columns by data type; **.tolist()** converts the resulting Index into a plain Python list. This is how the notebook automatically separates "things StandardScaler should touch" from "things OneHotEncoder should touch" without hardcoding column names.

```
df.isna().sum()
```

Counts missing values per column — this is how **total_bedrooms** gets flagged.

```
for col in df.columns:
    print(df[col].value_counts().head(20))
```

This loop is broader than just a missing-value check. It catches **disguised** missing data (placeholder values like -999 that aren't NaN but mean "missing"), reveals how unbalanced the `ocean_proximity` categories are, and helps spot capped or repeated numeric values — this is actually how the house-value cap gets noticed.

```
duplicate_mask = df.duplicated()
num_duplicates = duplicate_mask.sum()
```

duplicated() returns True for any row that's an exact copy of an earlier one; summing the boolean Series counts them.

Visualizations

- **sns.countplot** — bar chart of category frequencies (for `ocean_proximity`)
- **sns.histplot(..., kde=True)** on the target — histogram + smoothed density curve, revealing the right-skew and the value cap
- **plt.subplots(3, 3, figsize=(8,6)) + axes.flatten()** — creates a 3×3 grid of 9 subplot "slots" and flattens the 2D array into 1D so a single index can loop through it; used for both the histogram grid and boxplot grid
- **sns.boxplot** per column — shows median, IQR box, and outlier points beyond the whiskers at a glance
- **df[num_cols].corr()** — pairwise Pearson correlation matrix; **sns.heatmap(..., center=0)** colors it so 0 correlation is always white, making positive (red) vs negative (blue) instantly visible
- **corr_with_target = ...corr()[TARGET_COL].sort_values(ascending=False)** — slices just the target's row from that matrix and ranks every feature by linear correlation strength — this is how `median_income` gets identified as the strongest predictor

Key insights from EDA

- Dataset has numeric features plus one categorical feature (ocean_proximity)
- Only total_bedrooms has missing values
- Target (median_house_value) is right-skewed and capped
- Several features show strong skew and outliers
- median_income is the strongest predictor
- High multicollinearity among room and population features

4. Data Preprocessing

```
X = df.drop(columns=[TARGET_COL])
y = df[TARGET_COL]
```

X is every column except the target (the features); **y** is just the target.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE
)
```

Randomly splits both **X** and **y** together (rows stay aligned) into 80% train / 20% test. The fixed `random_state` means the same split happens every time the notebook is rerun.

```
numerical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

For numeric columns: fill missing values with the **median** (robust to skew and outliers — better than the mean here since several features are skewed), then scale. Scaling matters a lot for LinearRegression / Ridge / Lasso, since their coefficients and regularization penalties are scale-sensitive; tree models don't strictly need it, but it doesn't hurt them.

```
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
```

For categorical columns: fill missing values with the **mode**, then one-hot encode. **handle_unknown="ignore"** means if a category shows up later that the encoder never saw during training, it gets encoded as all-zeros instead of raising an error.

```
preprocess = ColumnTransformer(transformers=[
    ("num", numerical_transformer, numerical_features),
    ("cat", categorical_transformer, categorical_features)
])
```

Routes each column list through its matching mini-pipeline and concatenates the outputs into one feature matrix. This single **preprocess** object gets reused identically inside every model's pipeline below — that's what guarantees a fair, apples-to-apples comparison between models.

5. Baseline Model (No CV, No Tuning)

```
baseline_pipe = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", LinearRegression())
])
baseline_pipe.fit(X_train, y_train)
```

One `.fit()` call learns the imputer/scaler/encoder statistics *and* the regression coefficients — all from `X_train` only. Calling `.predict()` later on `X_test` applies those already-learned statistics to transform the test data; it does not refit anything on test data. That's the leak-proof part.

```
train_baseline_pred = baseline_pipe.predict(X_train)
test_baseline_pred = baseline_pipe.predict(X_test)

train_baseline_rmse = root_mean_squared_error(y_train, train_baseline_pred)
train_baseline_mae = mean_absolute_error(y_train, train_baseline_pred)
train_baseline_r2 = r2_score(y_train, train_baseline_pred)

test_baseline_rmse = root_mean_squared_error(y_test, test_baseline_pred)
test_baseline_mae = mean_absolute_error(y_test, test_baseline_pred)
test_baseline_r2 = r2_score(y_test, test_baseline_pred)
```

RMSE / MAE / R^2 are computed on **both** train and test. Comparing the two is how overfitting (train much better than test) or underfitting (both poor) gets caught.

6. Model Selection & Cross-Validation

```
models = {
    "LinearRegression": LinearRegression(),
    "Ridge": Ridge(random_state=RANDOM_STATE),
    "Lasso": Lasso(random_state=RANDOM_STATE, max_iter=10000),
    "RandomForest": RandomForestRegressor(),
    "HistGB": HistGradientBoostingRegressor()
}

cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
```

KFold shuffles the training rows first (important — otherwise folds could be biased if rows happen to be sorted by location), then splits into 5 equal chunks. The fixed seed means every model in the loop below is evaluated on the exact same 5 folds, which is what makes the comparison fair.

```
scoring = {
    "rmse": "neg_root_mean_squared_error",
    "mae": "neg_mean_absolute_error",
    "r2": "r2"
}
```

sklearn's scoring convention is "higher is better," so error metrics are stored as **negative** values — the sign has to be flipped back manually, which is why `-scores["test_rmse"].mean()` shows up later.

```

rows = []
for name, model in models.items():
    pipe = Pipeline(steps=[("preprocess", preprocess), ("model", model)])
    scores = cross_validate(pipe, X_train, y_train, cv=cv, scoring=scoring, n_jobs=1)
    rows.append({
        "model": name,
        "cv_rmse": -scores["test_rmse"].mean(),
        "cv_mae": -scores["test_mae"].mean(),
        "cv_r2": scores["test_r2"].mean()
    })

cv_results = pd.DataFrame(rows).sort_values("cv_rmse")

```

For each model: wrap it with the same preprocessing, then **cross_validate** internally (a) fits a fresh copy of the pipeline on 4 folds, (b) scores it on the 5th held-out fold, (c) rotates which fold is held out, 5 times total. It returns arrays such as `scores["test_rmse"]` — one value per fold — which then get averaged. **n_jobs=1** runs folds one at a time on a single core (sequential).

Turning the 5 result dicts into a DataFrame sorted by `cv_rmse` puts the lowest-error (best) model on top — which is how **HistGradientBoostingRegressor** gets crowned the winner.

7. Hyperparameter Tuning

```

hgb_pipe = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", HistGradientBoostingRegressor(random_state=RANDOM_STATE))
])

param_grid = {
    "model__learning_rate": [0.03, 0.05, 0.1],
    "model__max_depth": [None, 3, 6],
    "model__max_leaf_nodes": [15, 31, 63],
    "model__min_samples_leaf": [20, 50, 100],
    "model__l2_regularization": [0.0, 0.1, 1.0]
}

```

The **model__** prefix is required because the estimator is nested inside a Pipeline under the step name "model" — that's how GridSearchCV knows which step's parameter to set. With 5 hyperparameters × 3 values each, that's **3⁵ = 243 combinations**.

- **learning_rate** — how strongly each new tree corrects previous errors (smaller = slower but often more accurate)
- **max_depth** — max tree depth (None here means complexity is controlled by `max_leaf_nodes` instead)
- **max_leaf_nodes** — caps total leaves per tree, directly limiting model complexity
- **min_samples_leaf** — minimum samples per leaf; higher means smoother, more regularized predictions
- **l2_regularization** — shrinks leaf output values toward zero to fight overfitting

```

grid = GridSearchCV(
    estimator=hgb_pipe,
    param_grid=param_grid,
    cv=cv,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1,
    verbose=1,
)
grid.fit(X_train, y_train)

```

Runs full 5-fold CV for every combination — $243 \times 5 = 1,215$ total model fits. **n_jobs=-1** (unlike the **n_jobs=1** used earlier) uses all CPU cores in parallel, since this step is far heavier computationally. **verbose=1** prints progress. **grid.best_score_** and **grid.best_params_** give the best (sign-flipped back to positive) RMSE and the winning hyperparameter combination.

8. Retraining With Best Parameters

```

hgb_best = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", HistGradientBoostingRegressor(
        l2_regularization=0.1,
        learning_rate=0.1,
        max_depth=None,
        max_leaf_nodes=63,
        min_samples_leaf=20
    ))
])
hgb_best.fit(X_train, y_train)

```

Hardcodes the winning hyperparameters from the grid search into a fresh model and fits it on the entire training set, with no further CV happening — this is the final model that everything downstream uses.

Side note: GridSearchCV already refits a `best_estimator_` for you automatically by default. Rebuilding it manually here is functionally equivalent — just a more explicit, readable way of getting the same result.

9. Final Evaluation

Same RMSE / MAE / R^2 pattern as the baseline section, now computed for the tuned model on both train and test sets — letting you compare improvement over LinearRegression and check the train-vs-test gap for overfitting.

```

residuals = y_test - test_final_pred

plt.scatter(test_final_pred, residuals, s=10)
plt.axhline(0)
plt.title("Residuals vs Predictions")

```

residuals = actual minus predicted (positive = under-predicted, negative = over-predicted). Plotting residuals against predictions: ideally you want a random, structureless cloud centered on zero across the whole price range. Any visible pattern — a funnel shape, or residuals trending in one direction at high prices — signals systematic error in that range, which is very plausible here given the value cap found in EDA.

The histogram of residuals checks whether errors are roughly symmetric / bell-shaped around zero (good) or skewed (a sign of bias).

10. Building a Predictive System

```
def predict_house_price(model, longitude, latitude, housing_median_age,
                        total_rooms, total_bedrooms, population,
                        households, median_income, ocean_proximity) -> float:
    new_row = pd.DataFrame([
        "longitude": longitude,
        "latitude": latitude,
        "housing_median_age": housing_median_age,
        "total_rooms": total_rooms,
        "total_bedrooms": total_bedrooms,
        "population": population,
        "households": households,
        "median_income": median_income,
        "ocean_proximity": ocean_proximity
    ])
    return float(model.predict(new_row)[0])
```

- **new_row** has to be a DataFrame (not a plain list of numbers) because ColumnTransformer selects columns **by name**, not position.
- **model.predict()** always returns an array even for a single row, so [0] grabs the lone value; float(...) converts it from a numpy scalar to a plain Python float for clean printing.
- The fact that total_bedrooms can safely be np.nan is the real payoff of building everything into one Pipeline — the SimpleImputer step inside it handles that automatically at inference time, with zero extra code.

Double-check this in your own run: the function builds new_row using snake_case keys like housing_median_age — make sure that matches X_train.columns exactly (case and underscores), since ColumnTransformer matches columns by exact name and a mismatch would error or silently misroute the value. Also, the example inference cell appears twice in the original notebook — that looks like a re-run leftover rather than something intentional.

Ways to Improve Performance

- Create ratio features: rooms_per_household, bedrooms_per_room, population_per_household
- Try a log transformation on the target variable
- Tune max_iter and max_bins for HistGradientBoosting
- Try advanced boosting models like XGBoost or LightGBM
- Do error analysis by location or price range instead of relying on aggregate metrics alone
- Use spatial or grouped validation for location-based data